Image Processing

You are not logged in.

Please Log In for full access to the web site.
Note that this link will take you to an external site (`https://shimmer.mit.edu`) to authenticate, and then you will be redirected back to this page.

Table of Contents

Welcome

Welcome to 6.101! As this is our first lab, part of the time will be spent helping you familiarize yourself with the structure of a 6.101 lab and with the associated infrastructure. For that reason, and because of a compressed timetable compared to a normal lab, this lab is substantially smaller in terms of scope and scale than a typical lab.

1) Infrastructure

1.1) Necessary Software

Completing and submitting 6.101 assignments will require you to have a few pieces of software installed on your machine. Please follow our instructions for getting set up for 6.101 on your operating system of choice before proceeding.

This lab will also use the Pylint library, which analyzes Python code for common bugs and style issues. Note that Pylint's rating of your code will be discussed as part of your checkoff for this lab. See this page for instructions for installing Pylint (note that it's usually a good idea to use `pip3` instead of `pip`; if that doesn't work, try `python3 -m pip`).

You should also install Ruff, an auto-formatting tool for Python code. See this page for installation instructions. If you have any trouble installing, just ask, and we'll be happy to help you get set up.

1.2) The Command Line

Throughout 6.101, our instructions for labs will often refer to using your computer's *command line* (or "*shell*" or "*terminal*") to run programs. We don't expect you to be familiar with using the command line already; rather, we'll try our best to explain exactly what needs to be done when you do need to use the command line. However, you may find it helpful to familiarize yourself with some command-line basics before continuing on.

Although learning to use the command line is well worth the effort in the long run, it can be daunting when you're first getting started, so don't worry if things don't come naturally at first. Of course, feel free to ask at open lab hours or

office hours (or via e-mail at `6.101-help@mit.edu`) if you need help!

1.3) Running Tests

For each of our labs, we will include a file called `test.py` which contains several test cases designed to assess the correctness of your code for the lab.

Running `pytest test.py` will run all of the test cases in the file and show you the results. You can also modify this behavior (for example, to run only a subset of the test cases); the process for this is discussed in this section (about `pytest`) in the notes about the command line.

Note that some of the smaller tests might be useful not only for testing but also for help with understanding the specification for a given piece of code.

1.4) Submitting Code

As mentioned above, we will distribute a suite of tests with each lab, and you can use this file to test your code for correctness as often as you like.

Once your code passes these test cases on your machine, you should submit your code to the server to be checked, using the `6.101-submit` script, which will run your code through all of the tests for a lab (possibly including some tests that were *not* included in the distributed `test.py` file). More detailed instructions for submitting your code are available farther down this page.

1.5) Checkoff

For each lab, you must also complete an associated "checkoff," which is a brief conversation with a staff member about your code. At the checkoff, we will ask some questions about your code and also provide some feedback on style. The checkoff can also be a good opportunity to learn new Python tricks and alternative ways of approaching the lab!

Note that checkoffs will become available only after the associated lab has come due. The checkoffs themselves generally come due at 10pm on the Wednesday after the code submission came due.

Once you are ready (and the checkoff is available), come to any open lab time and use the help queue to request a checkoff.

Note that your checkoff will be based on the most recent code you have submitted, so if you make stylistic changes, etc., it is worth resubmitting your updated code so that we can discuss it during the checkoff. Per the lateness policy, submitting style changes after the deadline does not incur any lateness penalty, so long as all previous test cases continue to pass.

2) Preparation

This lab assumes you have Python 3.13 or later installed on your machine (3.14 recommended).

This lab will also use the `pillow` library, which we'll use for loading and saving images. See this page for instructions for installing pillow (note that it's usually a good idea to run `pip3` instead of `pip`; if that doesn't work, try `python3 -m pip`). If you have any trouble installing, just ask, and we'll be happy to help you get set up.

The following file contains code and other resources as a starting point for this lab: `image_processing.zip`

Your changes for this lab should be made to `lab.py`, which you will submit at the end of this lab. Importantly, you

should not add any imports to the file, nor should you use the `pillow` module for anything other than loading and saving images (which are already implemented for you).

Your raw score for this lab will be counted out of 5 points. Your score for the lab is based on:

- correctly answering the questions on this page (2 points)
- passing the tests in `test.py` (2 points)
- a brief "checkoff" conversation with a staff member about your code (1 point)

All of the questions on this page, including your code submission, are due at 5:00pm on Friday, 06 February. Checkoffs are due at 10:00pm on Wednesday, 11 February. **It is a good idea to submit your lab early and often so that if something goes wrong near the deadline, we have a record of the work you completed before then.** We recommend submitting your lab to the server after finishing each substantial portion of the lab.

3) Introduction

In this lab, you will build a few tools for manipulating digital images, akin to those found in image-manipulation toolkits like Photoshop and GIMP. Interestingly, many classic image filters are implemented using the same ideas we'll develop over the course of this lab.

For this week, we will focus on greyscale images only, but we'll revisit these ideas in next week's lab.

3.1) Digital Image Representation and Color Encoding

Before we can get to *manipulating* images, we first need a way to *represent* images in Python. While digital images can

be represented in myriad ways, the most common has endured the test of time: a rectangular mosaic of *pixels* -- colored dots, which together make up the image. An image, therefore, can be defined by specifying a *width*, a *height*, and an array of *pixels*, each of which is a color value. This representation emerged from the early days of analog television and has survived many technology changes. While individual file formats employ different encodings, compression, and other tricks, the pixel-array representation remains central to most digital images.

For this lab, we will simplify things a bit by focusing on greyscale images. Each pixel's brightness is encoded as a single integer in the range `[0,255]` (1 byte could contain 256 different values), `0` being the deepest black, and `255` being the brightest white we can represent. The full range is shown below:

For this lab, we'll represent an image using a Python dictionary with four keys:

- `"mode"`: either `"greyscale"` or `"color"` (for this lab, this will always be `"greyscale"`)
- `"width"`: the width of the image (in pixels),
- `"height"`: the height of the image (in pixels), and
- `"pixels"`: a Python list of pixel brightnesses stored in row-major order (listing the top row left-to-right, then the next row, and so on)

For example, consider this 3×2 image (3 rows, 2 columns) below (enlarged here for clarity):

This image would be encoded as the following instance:

```
image = {
    "mode": "greyscale",
    "height": 3,
    "width": 2,
    "pixels": [0, 50, 50, 100, 100, 255],
}
```

3.2) Loading, Saving, and Displaying Images

We have provided three helper functions in `lab.py` which may be helpful for debugging: `print_values`, `load_image` and `save_image`. Each of these functions is explained via a docstring.

You do not need to dig deeply into the actual code in those functions, but it is worth taking a look at those docstrings and trying to use the functions to:

- load an image, and
- save it under a different filename.

You can then use an image viewer on your computer to open the new image to make sure it was saved correctly. The `print_values` function will work best on very small images, for example `test_images/centered_pixel.png`.

There are several example images in the `test_images` directory inside the lab's code distribution, or you are welcome

to use images of your own to test, as well.

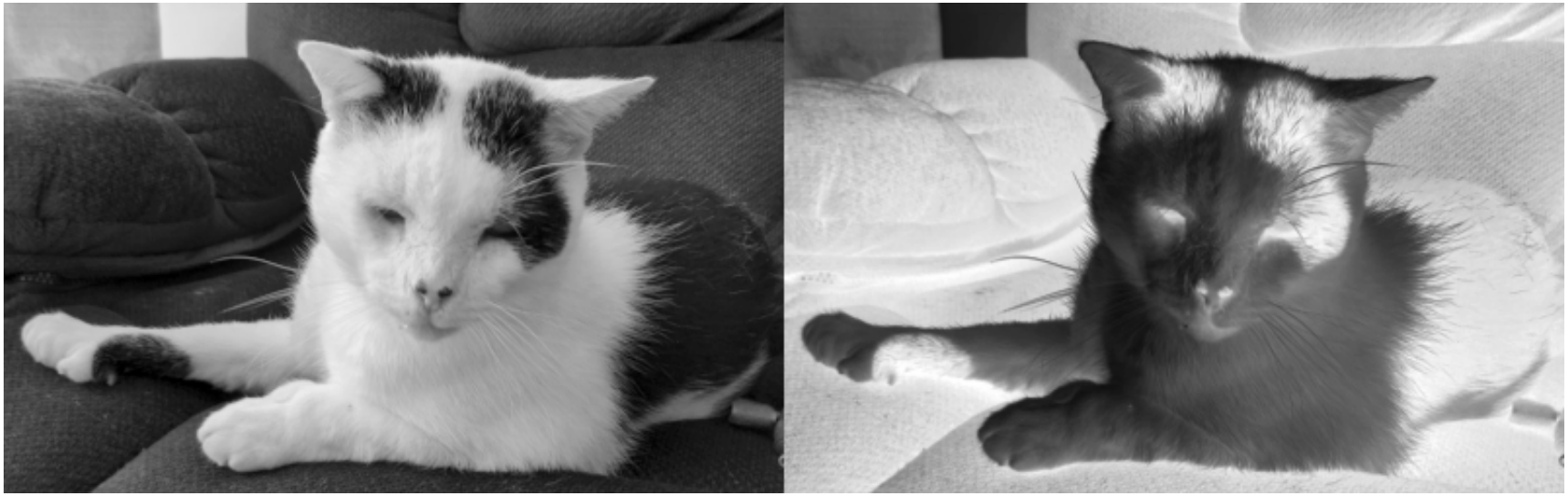
As you implement the various filters described below, these functions can provide a way to visualize your output, which can help with debugging, and they also make it easy to show off your cool results to friends and family.

Note that you should add code for loading, manipulating, and saving images under the `if __name__ == '__main__':` block in `lab.py`, which will be executed when you run `lab.py` directly (but not when it is imported by the test suite). As such, it is a good idea to get into the habit of writing code containing test cases of your own design within that block, rather than in the main body of the `lab.py` file.

4) Image Filtering via Per-Pixel Transformations

As our first task in manipulating images, we will look at an *inversion* filter, which reflects pixels about the middle grey value (`0` black becomes `255` white, `1` becomes `254`, and so on). For example, consider the following three images, showing Adam's cats. In each, the left side is the original image, while the right is an inverted version.

Notice how, in each example, the darkest spots in the input become the brightest spots in the output and *vice versa*, while grey spots in the input result in similar (but not necessarily identical) grey spots in the output.



Most of the implementation of the inversion filter has been completed for you (it is invoked by calling the function called `inverted` on an image), but some pieces have not been implemented correctly. Your task in this part of the lab is to fix the implementation of the inversion filter by finding and fixing errors in the `get_pixel`, `set_pixel`, and `inverted` functions so that they work as described.

Before you do that, however, let's add a small test case to `test.py`. We'll start by figuring out (by hand) what output we should expect in response to a given input and writing a corresponding test case.

Let's start with a 1×4 image that is defined with the following parameters:

- mode: `'greyscale'`
- height: `1`
- width: `4`

- pixels: `[22, 88, 136, 197]`

> If we were to run this image through a working inversion filter, what would the expected output be? Compute this result by hand. Then, in the box below, enter a Python list representing the expected value associated with the `pixels` key in the resulting image:

4.1) Adding a Test Case

While we could just add some code using `print` statements to check that things are working, we'll also use this as an opportunity to familiarize ourselves a bit with the testing framework we are using in 6.101.

Let's try adding this test case to the lab's regular tests so that it is run when we run `test.py`. If you open `test.py` in a text editor, you will see that it is a Python file that makes use of the Python package `pytest` for unit testing.

Each function that has a name starting with `test` in `test.py` represents a particular test case.

Running `test.py` through `pytest` will cause Python to run and report on *all* of the tests in the file. However, you can make Python run only a subset of the tests by running, for example, the following command from a terminal[1] (not including the dollar sign):

```
$ pytest test.py -k load -v
```

This will run any test with "load" in its name (in this case, the lone test defined in the `test_load` function). If you run this command, you should get a brief report indicating that the lone test case passed. By modifying that last argument (`load` in the example above), you can run different subsets of tests.

You can add a test case by adding a new function to the `test.py` file. Importantly, for it to be recognized as a test case, its name must begin with the word `test`[2].

In the skeleton you were given, there is a trivial test case called `test_inverted_2`. Modify this function so that it implements the test from above (inverting the small 1×4 image). Within that test case, you can define the expected result as an instance of the hard-coded dictionary, and you can compare it against the result from calling the `inverted` function on the original image. To compare two images, you may use our `compare_images` function (you can look at some of the other test cases to get a sense of how to use it). Your test case should also check to make sure that the input image was not modified.

Once you have implemented your function, paste it into the box below (you do not need to include the other functions/code from `test.py`). When you submit the box below, we will check your test case against several implementations of `inverted`, some of which work and some of which don't.

```
def test_inverted_2():
    assert False

```

Now that you've tested this function, copy it back into your `test.py` (replacing the existing `test_inverted_2`) so that it is run every time we run `test.py`. Even once we've implemented this function, we should also expect the test case to fail when run on your own code (after all, we haven't finished fixing the `inverted` filter yet!), but we can expect it to pass once all the bugs in the inversion filter have been fixed. In that way, it can serve as a useful means of guiding our bug-finding process (i.e., as long as it isn't passing, there are still more bugs to find!).

Throughout the lab, you are welcome to (and may find it useful to) add your own test cases for other parts of the code as you are debugging `lab.py` and any extensions or utility functions you write. Any function that starts with the word `test_` will be interpreted by `pytest` as a test case.

4.2) Debugging

Now, work through the process of finding and fixing the errors in the code for the inversion filter. **The bugs that you fix will be discussed as part of your checkoff, so it's a good idea to add comments describing all of the bugs you found and fixed as you're working, as you'll be expected to talk about not only your fixed code but also the nature of the bugs you found in the original code.** Happy debugging!

When you are done and your code passes all the inverted test cases, run your inversion filter on the `test_images/bluegill.png` image, save the result as a PNG image, and upload it below (choose the appropriate file and click "Submit"). If your image is correct, you will see a green check mark; if not, you will see a red X.

> Inverted `bluegill.png`:
> `No file selected`

It's also a good idea to go ahead and submit your code to the server at this point. Remember to **submit early and often**, not only for this lab but for all labs in the future!

5) Image Filtering via Correlation

Next, we'll explore some slightly more advanced image-processing techniques involving an operation called *correlation*. Before you start writing code, we encourage you to read all of this section, answer the small concept questions, and make a plan for how you will implement this behavior.

Given an input image I and a kernel k, applying k to I yields a new image O (perhaps with non-integer pixels outside of the range [0, 255]), equal in height and width to I, the pixels of which are calculated according to the rules described by k.

The process of applying a kernel k to an image I is performed as a *correlation*: the brightness of the pixel at position (r, c) in the output image, which we'll denote as $O_{r,c}$ (with $O_{0,0}$ being the upper-left corner), is expressed as a linear combination of the brightnesses of the pixels around position (r, c) in the input image, where the weights are given by the kernel k.

As an example, let's start by considering a 3×3 kernel:

```
0   1   0
0   0   0
0   0   0
```

For the remainder of this lab, we're going to represent a kernel as a dictionary mapping `(r, c)` coordinates to the associated values, with `(0, 0)` representing the center value in the kernel (we will assume that both dimensions of our kernels are odd in length so that there is always an unambiguous center value), and only including non-zero values. So the kernel above, for example, would be represented as `{(-1, 0): 1}`.

When we apply this kernel to an image I, the brightness of each output pixel $O_{r,c}$ is a linear combination of the brightnesses of the 9 pixels nearest to (r, c) in I, where each input pixel's value is multiplied by the associated value in the kernel.

In general, for a 3×3 kernel k, we have:

```
out[r,c] = (
```

```
      (inp[r-1, c-1] * k[-1, -1]) + (inp[r-1, c] * k[-1, 0]) + (inp[r-1, c+1] *
k[-1, 1])
    + (inp[r, c-1] * k[0, -1])    + (inp[r, c] * k[0, 0])    + (inp[r, c+1] * k[0,
1])
    + (inp[r+1, c-1] * k[1, -1]) + (inp[r+1, c] * k[1, 0]) + (inp[r+1, c+1] * k[1,
1])
)
```

So for example, applying the kernel k above at the center pixel in row 2, column 2 in the 5×5 image I below, which contains arbitrary pixel values:

```
  35  40  41  45  50
  40  40  42  46  52
  42  46  50  55  55
  48  52  56  58  60
  56  60  65  70  75
```

results in the value 42 being placed in the output image at row 2, column 2 because:

```
42 = (40*0) + (42*1) + (46*0)
   + (46*0) + (50*0) + (55*0)
   + (52*0) + (56*0) + (58*0)
```

For kernels larger or smaller than 3×3, we follow a similar pattern, always with `inp[r, c]` aligned with `k[0, 0]` at the center of this process.

Consider one step of correlating an image with the following 3 by 3 kernel:

```
 0.00  -0.07   0.00
-0.45   1.20  -0.25
 0.00  -0.12   0.00
```

which would be represented by the following dictionary:

```
{(-1, 0): -0.07, (0, -1): -0.45, (0,0): 1.20, (0, 1): -0.25, (1, 0): -0.12}
```

Given a 3 by 3 image with the values below:

```
 80   53    99
129  127   148
175  174   193
```

What will be the value of the center pixel in the output image? Enter a single number in the box below. Note

that, although our input brightnesses were all integers in the range $[0, 255]$, this value will be a decimal number.

5.1) Edge Effects

Before we continue on with actually implementing the correlation operator, there is one additional wrinkle we need to consider, which is that the correlation requires us accessing pixel locations that are outside of the bounds of the original image.

For a specific example, consider the top left pixel at position $(0, 0)$. In this case, all of the pixels to the top and left of $(0, 0)$ are out-of-bounds, but they should still factor into the correlation.

Currently, though, accessing pixel locations outside of image causes things to break, so we'll need to adjust that.

There are a bunch of different ways we could handle these out-of-bounds pixels, but for now, we're going to handle things in a very specific way: we're going to modify `get_pixel` so that it accepts out-of-bounds locations but always returns `0` for those pixel values. In effect, we're going to treat the image as being infinitely-large, surrounded on all sides by `0` values.

5.2) Correlation

Throughout the remainder of this lab, we'll implement a few different filters, each of which involves computing at least one correlation. As such, we will want to have a nice way to compute correlations in a general sense (for an arbitrary

image and an arbitrary kernel).

Note, though, that the output of a correlation need **not** be a legal 6.101 image (pixels may be outside the `[0,255]` range or may be floats). Since we want our filters all to output valid images, after the correlation process is finished each filter will need to *clip* negative pixel values to 0 and values greater than 255 to 255, and to ensure that all values in the image are integers.

Because correlation and making an image have valid pixel values are common operations, we're going to implement them separately, rather than as a single function that performs both operations.

We have provided skeletons for these two helper functions (which we've called `correlate` and `round_and_clip_image`, respectively) in `lab.py`. For now, read through the docstrings associated with these functions; we'll come back to implementing them in a bit after talking through some of the important pieces.

5.3) Example Kernels

Many different interesting operations can be expressed as correlations (some examples can be seen below), and many scientific programs also use this pattern, so feel free to experiment.

5.3.1) Identity

```
0 0 0
0 1 0
0 0 0
```

```
{(0, 0): 1}
```

The above kernel represents an *identity* transformation: applying it to an image yields the input image, unchanged.

5.3.2) Translation

```
0 0 0 0 0
0 0 0 0 0
1 0 0 0 0
0 0 0 0 0
0 0 0 0 0
```

```
{(0, -2): 1}
```

The above kernel shifts the input image two pixels to the *right*, discards the rightmost two columns of pixels, and replaced the two left-most columns with `0`'s.

5.3.3) Average

```
0.0 0.2 0.0
```

```
0.2 0.2 0.2
0.0 0.2 0.0
```

```
{(-1, 0): 0.2, (0, -1): 0.2, (0, 0): 0.2, (0, 1): 0.2, (1, 0): 0.2}
```

The above kernel results in an output image, each pixel of which is the average of the 5 nearest pixels of the input.

5.4) Tiny Correlations

Given the 3×3 kernel k:

```
  0   0   0
  0   0   1
  0   0   0
```

```
k = {(0, 1): 1}
```

And the greyscale image I with 3 rows and 2 columns below:

```
  20  40
```

```
 60  80
100 120
```

If we were to correlate image I with kernel k (assuming that values outside the bounds of the image are all `0`), what would the expected output be? Compute this result by hand. Then, in the box below, enter a Python list representing the expected value associated with the `pixels` key in the resulting image:

You may wish to use these examples to help with debugging. You may also want to write a few test cases comprised of correlating the 11-by-11 `test_images/centered_pixel.png` or another simple image with a few different kernels of different sizes. You can use the kernels in the previous section to help test that your code produces the expected results.

Now that we have a sense of a reasonable structure for this code and some example test cases, it's time to implement `correlate`!

5.5) Check Your Results

When you have implemented your code and are confident that it is working, try running it on `test_images/pigbird.png` with the following 13×13 kernel (which is all zeros except for a single value):

```
0 0 0 0 0 0 0 0 0 0 0 0 0
```

```
0 0 0 0 0 0 0 0 0 0 0 0 0
1 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0
```

How would we represent this kernel as a dictionary? Type out the corresponding dictionary below:

Now, run `correlate` on the `pigbird` image and save the result as a PNG image. Take a look at that image; does it match your expectations?

Result:

`No file selected`

It's also a good idea to go ahead and submit your code to the server at this point. Remember to **submit early and often**!

6) Blurring and Sharpening

6.1) Blurring

For this part of the lab, we will implement a box blur, which will make a blurry version of the input image, and which can be implemented via correlation. For a box blur, the kernel is an $n \times n$ square of identical values that sum to 1.

Notice that we have provided an outline for a function called `blurred` in the lab distribution. Read the docstring and the outline for `blurred` to get a sense for how it should behave.

Before implementing it, create two additional test cases by filling in the definitions of `test_blurred_black_image` and `test_blurred_centered_pixel` with the following tests:

- Box blurs of a 6×5 image consisting entirely of black pixels, with two different kernel sizes. The output is trivially identical to the input.
- Box blurs for the `centered_pixel.png` image with two different kernel sizes. You should be able to compute the output manually.

These test cases can serve as a useful check as we implement the box blur.

Once you have implemented your `test_blurred_black_image` function, paste it into the box below (you do not need to include the other functions/code from `test.py`).

```
def test_blurred_black_image():
    assert False

```

Once you have implemented your `test_blurred_centered_pixel` function, paste it into the box below (you do not need to include the other functions/code from `test.py`).

```
def test_blurred_centered_pixel():
    assert False

```

Finally, implement the box blur by filling in the body of the `blurred` function.

6.1.1) Check Your Results

When you are done and your code passes all the blur-related tests (including the ones you just created), run your blur filter on the `test_images/cat.png` image with a box-blur kernel size of 13, save the result as a PNG image, and upload it below to be checked for correctness:

Blurred `cat.png`:

No file selected

6.2) Sharpening

Next, we'll implement the opposite operation, known as a *sharpen* filter. The "sharpen" operation often goes by another name which is more suggestive of what it means: it is often called an *unsharp mask* because it results from subtracting an "unsharp" (blurred) version of the image from a scaled version of the original image.

More specifically, if we have an image (I) and a blurred version of that same image (B), the value of the sharpened image S at a particular location is:

$$S_{r,c} = 2I_{r,c} - B_{r,c}$$

One way we could implement this operation is by computing a blurred version of the image, and then, for each pixel, computing the value given by the equation above.

While you are not required to do so, it is actually possible to perform this operation with a single correlation (with an appropriately chosen kernel).

> **Check Yourself:**
>
> If we want to use a blurred version B that was made with a 3×3 blur kernel, what kernel k could we use to compute S in its entirety with a single correlation (i.e., without producing B at all)?

Implement the sharpen filter as a function `sharpened(image, kernel_size)`, where `image` is an image and

`kernel_size` denotes the size of the blur kernel that should be used to generate the blurred copy of the image. You are welcome to implement this as a single correlation or using an explicit subtraction, though if you use an explicit subtraction, make sure that you do not do any rounding until the end (the intermediate blurred version should not be rounded or clipped in any way).

Note that after computing the above, we'll still need to make sure that `sharpened` ultimately returns a valid 6.101 image, which you can do by making use of your helper function from earlier in the lab.

6.2.1) Check Your Results

When you are done and your code passes the tests related to sharpening, run your sharpen filter on the `test_images/python.png` image with a kernel of size 11, save the result as a PNG image, and upload it below to be checked:

> Sharpened `python.png`:
> `No file selected`

7) Code Submission

When you have tested your code sufficiently on your own machine, and you are happy with it stylistically, submit your final version of `lab.py` using the `6.101-submit` script. If you haven't already installed that script, see the instructions on this page.

The following command should submit the lab, assuming that the last argument `/path/to/lab.py` is replaced by the

location of your `lab.py` file:

```
$ 6.101-submit -a image_processing /path/to/lab.py
```

Running that script should submit your file to be checked. After submitting your file, information about the checking process can be found below:

A Python Error Occurred:

```
Error on line 27 of question tag.


ModuleNotFoundError: No module named 'ansi2html'
```

Please log in to see Hypothesis Profiler results.

8) Checkoff

Once you are finished with the code, you will need to come (in person) to any open lab time and add yourself to the queue asking for a checkoff in order to receive credit for the lab. **You must be ready to discuss your code in detail before asking for a checkoff.**

You should be prepared to demonstrate your code (which should be well-commented, should avoid repetition, and should make good use of helper functions; see the notes on style for more information). In particular, be prepared to discuss:

- What was wrong with the original code for `inverted`? For reference, here is the original code:

▶ Original `inverted` code

- Your implementation of correlation.
- Your implementation of the blur effect and the associated test cases.
- Your implementation of the `sharpen` operation.

> *You have not yet received this checkoff. When you have completed this checkoff, you will see a grade here.*

9) What Comes Next?

In next week's lab, we're going to continue to explore some of these ideas, implementing some additional filters and expanding our little codebase to handle color images as well.

There are a lot of interesting classes at MIT that explore ideas related to this lab. 6.3000 (Signal Processing) is a great option for the future, as a way to get a different perspective and a deeper understanding of how filters like these behave. 6.8371 (Digital and Computational Photography) is another class that is worth considering, perhaps later in your career, where you get an opportunity to learn about (and implement) a number of other interesting image-processing applications.

Footnotes

[1] Note that you won't be able to run this command from within Python; rather, it should be run from a terminal. Our readings on using the command line should get you started. If you want to try it out and are having trouble, we're happy to help in open lab hours!

[2] `pytest` has many other features, such as grouping tests or creating test classes whose methods form groups of tests. Check out its documentation if you are interested in learning about these features.